Project AETHER

Autonomous Constellation Manager for
Orbital Debris Avoidance and Station-Keeping

National Space Hackathon 2026 — Technical Report

Pranjal Saxena

March 2026

Contents

1 Executive Summary **2**

2 System Architecture **2**
2.1 Overview . . . 2
2.2 Module Decomposition . . . 2
2.3 Data Flow . . . 3

3 Numerical Methods **3**
3.1 Equations of Motion (J2 Perturbation) . . . 3
3.2 4th-Order Runge-Kutta Integration . . . 3
3.3 Performance Optimization . . . 3

4 Spatial Optimization: Conjunction Detection **3**
4.1 The $O(N^2)$ Problem . . . 3
4.2 KD-Tree Screening ($O(N \log N)$) . . . 4
4.3 Coarse-Then-Fine TCA Search . . . 4

5 Maneuver Planning and Execution **4**
5.1 RTN Frame and Evasion Strategy . . . 4
5.2 Fuel Mass Depletion (Tsiolkovsky) . . . 4
5.3 Constraint Validation . . . 5
5.4 Blind Conjunction Handling . . . 5
5.5 Station-Keeping and Recovery . . . 5
5.6 End-of-Life Management . . . 5

6 Ground Station Network and Communication **5**
6.1 Line-of-Sight Geometry . . . 5
6.2 ECI to ECEF Conversion . . . 5

7 Frontend: Orbital Insight Dashboard **6**

8 Technology Stack **6**

9 Evaluation Criteria Mapping **6**

10 Conclusion **7**

1 Executive Summary

Project AETHER (**A**utonomous **E**vasion, **T**racking, **H**euristic **E**ngine for **R**esilient orbits) is a full-stack Autonomous Constellation Manager (ACM) that manages a fleet of 50+ LEO satellites navigating a hazardous debris environment of 10,000+ tracked objects. The system ingests high-frequency orbital telemetry, predicts conjunctions up to 24 hours ahead using KD-tree spatial indexing, autonomously computes and executes collision-avoidance and station-keeping maneuvers, and presents real-time situational awareness through a 3D operational dashboard.

Key technical achievements:

- J2-perturbed orbital propagation via 4th-order Runge-Kutta with Numba JIT — 10,000 objects propagated in $\sim$0.1 s
- $O(N \log N)$ conjunction screening via `scipy.spatial.KDTree` with coarse-then-fine TCA refinement
- Constraint-validated maneuver scheduling enforcing cooldown, LOS, signal delay, fuel budget, and max-Δv limits
- Blind-conjunction handling: pre-uploading evasion sequences before satellite enters communication blackout
- Three.js 3D globe visualization rendering 10,000+ debris at 60 FPS via WebGL `InstancedMesh`

2 System Architecture

2.1 Overview

The ACM is a containerized application (Docker, `ubuntu:22.04`) exposing a RESTful API on port 8000. It comprises a Python/FastAPI backend for physics computation and autonomous decision-making, and a React/Three.js frontend for visualization.

2.2 Module Decomposition

Table 1: Backend module responsibilities

Module	Responsibility
`physics/propagator.py`	RK4 numerical integrator with J2 perturbation; batch-parallel via Numba
`physics/coordinates.py`	ECI↔ECEF↔Geodetic transformations; GMST computation
`physics/maneuver.py`	RTN frame construction; evasion/recovery Δv planning; Tsiolkovsky fuel equation
`engine/state_manager.py`	In-memory state store using NumPy arrays for vectorized operations
`engine/conjunction.py`	KD-tree spatial screening + coarse/fine TCA search
`engine/scheduler.py`	Constraint-validated maneuver queue; auto-evasion; EOL management
`engine/ground_stations.py`	Line-of-sight computation; visibility window prediction
`engine/station_keeping.py`	Nominal slot drift monitoring; uptime tracking; recovery scheduling
`utils/logger.py`	Structured JSON event logging for all maneuvers and detections

2.3 Data Flow

1. Grader sends object states via `POST /api/telemetry`
2. State manager ingests and indexes all objects
3. Conjunction detector screens pairs via KD-tree, refines close approaches
4. Scheduler auto-queues evasion burns for critical CDMs, validates constraints
5. Grader advances time via `POST /api/simulate/step`
6. Propagator integrates all orbits; scheduler executes due burns
7. Station-keeping manager checks slot compliance, schedules recovery burns
8. Frontend polls `GET /api/visualization/snapshot` for real-time display

3 Numerical Methods

3.1 Equations of Motion (J2 Perturbation)

All propagation uses the J2-perturbed two-body equations of motion in the ECI (J2000) frame as specified in the problem statement:

$$\ddot{\vec{r}} = -\frac{\mu}{|\vec{r}|^3}\,\vec{r} + \vec{a}_{J_2} \tag{1}$$

where $\mu = 398600.4418\ \mathrm{km}^3/\mathrm{s}^2$ and the J2 acceleration is:

$$\vec{a}_{J_2} = \frac{3}{2}\frac{J_2\,\mu\,R_E^2}{|\vec{r}|^5}\begin{pmatrix} x\left(\frac{5z^2}{|\vec{r}|^2} - 1\right) \\ y\left(\frac{5z^2}{|\vec{r}|^2} - 1\right) \\ z\left(\frac{5z^2}{|\vec{r}|^2} - 3\right) \end{pmatrix} \tag{2}$$

with $R_E = 6378.137$ km and $J_2 = 1.08263 \times 10^{-3}$.

3.2 4th-Order Runge-Kutta Integration

The state vector $\vec{y} = [\vec{r},\, \vec{v}]^T$ is integrated using the classical RK4 scheme:

$$\vec{y}_{n+1} = \vec{y}_n + \frac{h}{6}\left(\vec{k}_1 + 2\vec{k}_2 + 2\vec{k}_3 + \vec{k}_4\right) \tag{3}$$

with the standard evaluation points. Our default timestep is $h = 10\,\mathrm{s}$, chosen to balance accuracy against computational cost for LEO orbits with periods of $\sim$90 min.

3.3 Performance Optimization

All propagation kernels use **Numba JIT** compilation (`@njit`) to achieve near-C performance. Batch propagation uses `@njit(parallel=True)` with `prange` to distribute across CPU cores. This achieves:

- Single object, 1 orbit ($\sim$5400 s): $\sim$0.5 ms
- 10,000 objects, 3600 s step: $\sim$100 ms on 8 cores

4 Spatial Optimization: Conjunction Detection

4.1 The $O(N^2)$ Problem

Naïvely checking every satellite–debris pair costs $O(N_s \cdot N_d)$ per timestep. With $N_s = 50$ and $N_d = 10{,}000$, this is 500,000 distance computations per step, multiplied across hundreds of propagation steps for a 24-hour prediction horizon.

4.2 KD-Tree Screening ($O(N \log N)$)

We construct a `scipy.spatial.KDTree` over the debris positions and query a screening ball of radius $R_{\text{screen}} = 500\,\text{km}$ around each satellite. This reduces the candidate set from $O(N_d)$ to $O(k)$ where $k \ll N_d$ for each satellite.

4.3 Coarse-Then-Fine TCA Search

For each candidate pair surviving the KD-tree screen:

1. **Coarse pass**: Propagate both objects at 60 s intervals over 24 hours; record the time of minimum distance.
2. **Fine pass**: Re-propagate a $\pm 120\,\text{s}$ window around the coarse minimum at 1 s intervals to refine the Time of Closest Approach (TCA) and miss distance.

This two-phase approach avoids expensive fine-grained propagation for the vast majority of pairs that are not true threats.

Algorithm 1 Conjunction Assessment Pipeline

```
tree ← KDTree(debris_positions)
for each satellite s do
    nearby ← tree.query_ball_point(s.r⃗, R_screen)
    for each debris d in nearby do
        (t_min, d_min) ← coarse_search(s, d, Δt = 60s, T = 86400s)
        if d_min < 5 km then
            (t_TCA, d_TCA) ← fine_search(s, d, t_min ± 120s, Δt = 1s)
            Classify risk: CRITICAL (<100 m), RED (<1 km), YELLOW (<5 km)
        end if
    end for
end for
```

5 Maneuver Planning and Execution

5.1 RTN Frame and Evasion Strategy

Maneuvers are computed in the satellite's local Radial-Transverse-Normal (RTN) frame:

$$\hat{R} = \frac{\vec{r}}{|\vec{r}|}, \quad \hat{N} = \frac{\vec{r} \times \vec{v}}{|\vec{r} \times \vec{v}|}, \quad \hat{T} = \hat{N} \times \hat{R} \tag{4}$$

Our primary evasion strategy applies a **transverse burn** ($\hat{T}$ direction), which is the most fuel-efficient way to shift the satellite's along-track timing. The required Δv scales inversely with lead time, motivating early detection and scheduling.

5.2 Fuel Mass Depletion (Tsiolkovsky)

Every burn depletes propellant according to the rocket equation:

$$\Delta m = m_{\text{current}} \left(1 - e^{-\frac{|\Delta \vec{v}|}{I_{sp}\, g_0}}\right) \tag{5}$$

with $I_{sp} = 300\,\text{s}$ and $g_0 = 9.80665\,\text{m/s}^2$. The system dynamically tracks mass, meaning later burns on a lighter satellite are slightly more efficient.

5.3 Constraint Validation

Every maneuver command passes through 6 validation gates before acceptance:

Constraint	Implementation
Max Δv	$\lvert\Delta\vec{v}\rvert \leq 15\,\mathrm{m/s}$ per burn
Signal delay	Burn time $\geq$ current time $+10\,\mathrm{s}$
Fuel budget	Tsiolkovsky mass check against remaining propellant
Cooldown	$600\,\mathrm{s}$ minimum between burns on the same satellite
Ground station LOS	Elevation angle $\geq$ station mask at upload time
Satellite existence	Valid ID check in state registry

5.4 Blind Conjunction Handling

If a critical conjunction is predicted while the satellite is in a communication blackout zone:

1. The system propagates the satellite forward to find the next ground station contact window.
2. If the contact window occurs *before* the TCA, the evasion sequence is scheduled for upload during that window.
3. If no contact window exists before TCA, the maneuver cannot be executed (logged as unresolvable).

5.5 Station-Keeping and Recovery

After evasion, the system monitors drift from the nominal orbital slot (10 km tolerance). When drift is detected and no conflicting maneuver is pending, a phasing recovery burn is automatically scheduled in the prograde/retrograde direction to adjust the orbital period and allow the satellite to drift back.

5.6 End-of-Life Management

When fuel drops below 5% ($\leq 2.5\,\mathrm{kg}$), a graveyard orbit raise of $\sim$25 km is autonomously scheduled to prevent the satellite from becoming uncontrollable debris.

6 Ground Station Network and Communication

6.1 Line-of-Sight Geometry

For each of the 6 ground stations, the elevation angle of a satellite is computed as:

$$\sin(\varepsilon) = \frac{(\vec{r}_{\mathrm{sat}} - \vec{r}_{\mathrm{gs}}) \cdot \hat{u}_{\mathrm{gs}}}{|\vec{r}_{\mathrm{sat}} - \vec{r}_{\mathrm{gs}}|} \tag{6}$$

where $\hat{u}_{\mathrm{gs}}$ is the local "up" unit vector at the ground station (its ECEF position normalized). Visibility requires $\varepsilon \geq \varepsilon_{\mathrm{min}}$ (station-specific mask angle from 5° to 15°).

This formulation naturally accounts for Earth curvature: a satellite behind Earth yields a negative or sub-threshold elevation.

6.2 ECI to ECEF Conversion

The ECI-to-ECEF rotation uses Greenwich Mean Sidereal Time:

$$\theta_{\mathrm{GMST}} = 280.46061837^\circ + 360.98564736629 \cdot (JD - 2451545.0) + \ldots \tag{7}$$

applied as a z-axis rotation to transform satellite positions from the inertial to Earth-fixed frame for ground station geometry.

7 Frontend: Orbital Insight Dashboard

The frontend is a single-page React application with four PS-required visualization modules:

1. **3D Globe (Ground Track View)**: Three.js Earth sphere with NASA Blue Marble texture, satellite markers, debris cloud via `InstancedMesh` ($\sim$1 draw call for 10K points), ground station markers, and threat lines. Supports orbit controls (rotate, zoom).
2. **Conjunction Bullseye Plot**: SVG polar chart with the selected satellite at center. Radial distance encodes TCA; angle encodes approach vector. Color indicates risk level (red/amber/green).
3. **Fleet Fuel Heatmap**: 10×5 grid of color-coded cells per satellite showing propellant percentage, plus a bar chart and min/avg/max summary.
4. **Maneuver Timeline (Gantt)**: Horizontal tracks per satellite showing evasion burns (red), recovery burns (cyan), and 600 s cooldown blocks (hatched), with a "NOW" marker.

Performance: The `InstancedMesh` approach renders 10,000 debris objects as a single GPU draw call. Combined with React Three Fiber's reconciler, the dashboard maintains 60 FPS with full interactivity.

8 Technology Stack

Layer	Technology	Purpose
API Framework	FastAPI + Uvicorn	Async REST API on port 8000
Physics	NumPy + Numba	Vectorized propagation with JIT compilation
Spatial Index	SciPy KDTree	$O(N \log N)$ conjunction screening
Frontend	React 18 + Vite	Component-based dashboard with HMR
3D Rendering	Three.js + R3F	WebGL Earth globe and orbital visualization
Charts	SVG + Recharts	Polar plots, fuel gauges, timelines
Containerization	Docker (ubuntu:22.04)	Reproducible deployment
Logging	JSON structured logs	Machine-readable event stream

9 Evaluation Criteria Mapping

Criterion	Weight	Our Approach
Safety Score	25%	Autonomous CRITICAL CDM evasion; 2 km safety margin; blind-conjunction pre-upload
Fuel Efficiency	20%	Transverse-first strategy; early detection reduces Δv; mass-aware Tsiolkovsky
Constellation Uptime	15%	Automatic recovery burns; drift monitoring; exponential penalty tracking
Algorithmic Speed	15%	KD-tree $O(N \log N)$; Numba JIT parallel propagation; coarse/fine TCA
UI/UX & Visualization	15%	3D WebGL globe; 60 FPS InstancedMesh; all 4 required modules
Code Quality & Logging	10%	Modular architecture; structured JSON logs; type-validated API

10 Conclusion

Project AETHER demonstrates that autonomous constellation management is achievable with efficient numerical methods and intelligent constraint handling. Our KD-tree spatial indexing eliminates the $O(N^2)$ bottleneck, Numba-accelerated propagation handles 10,000+ objects in real-time, and the six-gate maneuver validation pipeline ensures no burn violates the physical or operational constraints of the mission. The 3D operational dashboard provides the situational awareness needed for human-in-the-loop oversight while the autonomous engine handles the high-frequency decision-making that manual operations cannot scale to.